

Task 1.1	Reads the warehouse map from the text file into a two-dimensional list	Mark		Remarks
	Correctly locates the robot's row and column	1	0	file.close() at incorrect indent
	Returns Warehouse, RobotRow and RobotCol in the correct order	1	0	
	Correctly tests the function by displaying the Warehouse and robot's starting position	1	1	
	Sub-total	1	0	
		4	1	

Task 1.2	Correctly determines the destination row and column from Direction	1		
	Correctly checks whether the destination is outside the warehouse boundaries	1		
	Correctly checks whether the destination is a shelf (#)	1		
	When the move is invalid, displays "Invalid move", returns the original position and does not modify the Warehouse	1		
	When move is successful, changes previous robot position to C and destination to R, and returns new robot position	1		
	Tests using U, D, R and L and displays Warehouse after each move attempt	1		
	Sub-total	6	0	no evidence

Task 1.3	Correctly examines only the four adjacent positions	1		
	Correctly identifies and counts warehouse boundaries	1		
	Correctly counts dirty floors, clean floors and shelves	1		
	Correctly returns the four counts and tests the ScanNearby() function by displaying them	1		
	Sub-total	4	0	no evidence

Task 1.4	Repeatedly accepts commands until Q and displays "Invalid command" for an invalid command	1		
	Correctly processes U, D, R and L by calling MoveRobot() and updating RobotRow and RobotCol using the returned values	1		
	Correctly processes S by calling ScanNearby() and displaying the four returned values; S does not affect either counter	1		
	Correctly updates SuccessfulMoves or InvalidMoves and displays the Warehouse after each movement	1		
	Correctly determines the final number of dirty tiles and displays Successful moves, Invalid moves and Dirty tiles remaining	1		
	Sub-total	5	0	no evidence

Task 2.1	Reads REGISTRATIONS.txt and stores the records correctly in Registrations as a list of lists	1	0	
	Implements MergeSort() recursively with an appropriate base case	1	0	
	Correctly calculates the midpoint/divides the current sub-list	1	0	
	Correctly makes the recursive call for the left sub-list	1	0	
	Correctly makes the recursive call for the right sub-list	1	0	
	Correct comparison during merge	1	0	
	Correctly merges elements while both sub-lists contain elements	1	0	
	Correctly copies remaining elements from left/right sub-list	1	0	
	Sorts by Course Code in ascending alphabetical order	1	0	
	When Course Codes are equal, orders by Student Name in ascending alphabetical order	1	0	
	Displays the first ten registrations before and after sorting	1	0	
	Sub-total	11	0	

Task 2.2	Implements an iterative binary search, repeatedly calculating the middle index	1	0	no calculation of middle
	Correctly changes the search boundaries based on comparison with CourseCode	1	0	no boundaries update
	Returns the index of a matching registration when found	1	0	
	Returns -1 when no match exists and correctly tests/displays results for CS101 and CS999	1	0	
	Sub-total	4	0	

Task 2.3	Calls BinarySearch() with the specified CourseCode	1	1	
	Correctly handles -1 and displays Course not found	1	0	
				Incomplete logic to locate the matching group from the returned index
	Uses the index returned by BinarySearch() to locate the matching group rather than starting from index 0	1	0	
	Processes consecutive matching records in the group	1	0	
	Correctly displays all matching students, including when the returned index is in the middle of the group	1	0	
	Sub-total	5	1	

	Sub-total	11	0	
Task 2.2	Implements an iterative binary search, repeatedly calculating the middle index	1	0	no calculation of middle
	Correctly changes the search boundaries based on comparison with CourseCode	1	0	no boundaries update
	Returns the index of a matching registration when found	1	0	
	Returns -1 when no match exists and correctly tests/displays results for CS101 and CS999	1	0	
	Sub-total	4	0	
Task 2.3	Calls BinarySearch() with the specified CourseCode	1	1	
	Correctly handles -1 and displays Course not found	1	0	
	Uses the index returned by BinarySearch() to locate the matching group rather than starting from index 0	1	0	Incomplete logic to locate the matching group from the returned index
	Processes consecutive matching records in the group	1	0	
	Correctly displays all matching students, including when the returned index is in the middle of the group	1	0	
	Sub-total	5	1	
Task 2.4	Correctly counts the number of registrations for each distinct Course Code	1		
	Correctly displays the number of students for each Course Code	1		
	Correctly determines and displays the total number of different Course Codes	1		
	Correctly determines the largest class size	1		
	Correctly identifies and displays all Course Codes with the largest class size	1		
	Sub-total	5	0	no evidence
Task 3.1	Creates a Flask web application and connects to WORKSHOPS.db	1	1	
	Uses SQL to retrieve all records from the Workshop table, sorted in ascending order of Title	1	1	
	Passes the retrieved records to INDEX.html for rendering	1	1	workshops is passed to the template and used in the loop
	Provides navigation to the workshop registration page and workshop registration report page	1	1	Links to /REGISTER and /REPORT are provided
	Saved web page output shows all workshop records are displayed in the correct order with the required information	1	0	
	Sub-total	5	0	Required .py file not submitted; only .ipynb file provided.
Task 3.2	Uses SQL to retrieve all students from the Student table	1	0	student_id is undefined in the GET branch
	Uses SQL to retrieve all workshops from the Workshop table	1	0	workshop_id is undefined in the GET branch
	Uses the retrieved data to populate the two selection lists in REGISTER.html	1	1	students and workshops passed to template
	Correctly obtains the selected StudentID and WorkshopID when the form is submitted	1	0	should use request.form[...] or request.form.get(...)
	Determines the number of students already registered for the selected workshop and compares it with MaxPlaces	1	0	get.db is invalid, so the query cannot execute
	Determines whether a registration already exists for the selected student and workshop	1	1	Attempts to determine whether selected student has already registered for the selected workshop
	Prevents insertion and displays an appropriate message when the workshop is full	1	1	compares current registrations with MaxPlaces and prevents insertion when full
	Prevents insertion and displays an appropriate message when the registration already exists	1	0	preceding database operations do not work, hence unable to evaluate conditions
	Inserts new registration with current system date as RegistrationDate, commits the change and displays a confirmation message	1	0	INSERT unsuccessful; Registrations is wrong table; datetime.now() invalid
	Provides saved output showing a successful registration	1	0	
	Provides saved output showing a workshop-full case and a duplicate-registration case	1	0	
	Sub-total	11	0	Required .py file not submitted; only .ipynb file provided.
Task 3.3	Writes SQL that retrieves all workshops, including workshops with no registrations	1	0	Uses INNER JOIN, so workshops with no registrations are excluded
	Uses SQL to determine the number of students registered for each workshop	1	0	COUNT(*) is correct, GROUP BY StudentID does not determine count required
	Sorts results in descending order of number registered, with ascending Title as tie-break	1	0	missing comma in ORDER BY
	Retrieves the data required by the provided REPORT.html and passes it to the template	1	1	data passed to template
	Determines the maximum number of registrations from the report data	1	0	workshop is undefined and no maximum registration count is determined
	Identifies all workshop titles whose number of registrations equals the maximum, including ties	1	0	no tie-handling logic

	Produces the workshop registration report page using the provided REPORT.html	1	1	correct render_template()
	Correctly supplies the registration count for each workshop to the template	1	1	SELECT attempt to produce studentcount using COUNT(*)
	Successfully runs the application and saves the report output	1	0	
	Saved output demonstrates the required report content, including workshop information and ordering	1	0	
	Sub-total	10	0	Required .py file not submitted; only .ipynb file provided.
Task 4.1	Declares class Presentation with a constructor that accepts the required parameters	1	1	
	Correctly initialises OrderNumber, TeamName, ProjectTitle and Category	1	1	
	Initialises NextPresentation to None	1	1	
	Correctly implements all five getter methods and setNext(P)	1	1	
	Correctly implements display() to display OrderNumber, TeamName, ProjectTitle and Category	1	1	
	Creates a Presentation object and calls display()	1	1	
	Sub-total	6	6	
Task 4.2	Declares class PresentationSchedule and initialises FirstPresentation to None	1	1	
	Implements getFirst() and setFirst(P) correctly	1	1	
	Declare AddPresentation(NewPresentation) and handles an empty schedule	1	0	No empty-list handling; current would be None
	Detects a duplicate OrderNumber and does not insert duplicate	1	0	Uses GetOrderNumber() instead of getOrderNumber()
	Correctly inserts NewPresentation before the first presentation	1	0	not implemented
	Uses recursion to traverse the schedule to find the correct insertion position	1	1	
	Correctly identifies when NewPresentation should be inserted between two presentations	1	1	
	Correctly uses Previous and Current during recursive traversal without losing the remainder of the list	1	0	
	Correctly inserts NewPresentation at the end of the schedule	1	1	
	Summary() correct traverses the presentation schedule	1	1	traverse using getNext()
	Calls display() to display each presentation	1	1	
	Correctly calculates the total number of presentations	1	1	use size
	Correctly displays the total number of presentations	1	1	
	Correctly counts the number of presentations for each category	1	1	
	Correctly displays the number of presentations for each category	1	0	Only the category name is printed, not its count
	Sub-total	15	10	
Task 4.3	Correctly initialises a PresentationSchedule object	1	1	
	Correctly creates the six Presentation objects and calls AddPresentation() for each in the order given	1	1	
	Calls Summary() and shows the output	1	0	no output, typo error in test_dict
	Sub-total	3	2	
Coding Style	Meaningful identifiers	1	1	
		1	1	
	Whitespaces	1	1	
		1	1	
	Comments	1	1	
		1	0	
	Sub-total	6	5	
TOTAL		100	25	